# Mooncake and Disaggregated Serving

Deep Dive into Kimi K3 — Chapter 43

Yin Yang

Foundation Books Project

# The one idea of this chapter

Separating prefill from decode turns cached state into a resource that moves; it becomes service only when identity, lifetime and admission all hold.

# The one idea of this chapter

Separating prefill from decode turns cached state into a resource that moves; it becomes service only when identity, lifetime and admission all hold.

Goal: follow one request, $R_B$, from prefix lookup through a protected handoff to a completed answer — and count it only if it meets the latency predicate.

# Roadmap

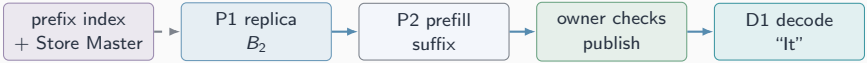
State with an owner

Moving and reusing state

Placement, admission and recovery

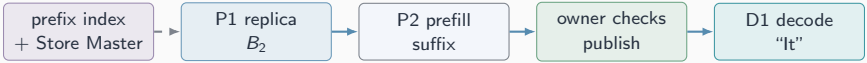
# State with an owner

# Identity, storage and a protected handoff



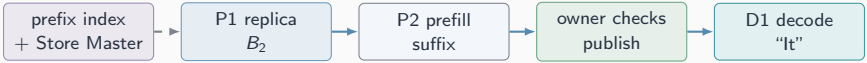

- Reuse needs a continuous chain of prefix-dependent block keys under one namespace, version and layout; $R_B$ reuses 8 tokens, $R_C$ only 4.

# Identity, storage and a protected handoff




- Reuse needs a continuous chain of prefix-dependent block keys under one namespace, version and layout; $R_B$ reuses 8 tokens, $R_C$ only 4.
- Metadata is not payload: allocate, write and mark readable are separate events, and every copy counts at every event (256 bytes/token here).

# Identity, storage and a protected handoff




- Reuse needs a continuous chain of prefix-dependent block keys under one namespace, version and layout; $R_B$ reuses 8 tokens, $R_C$ only 4.
- Metadata is not payload: allocate, write and mark readable are separate events, and every copy counts at every event (256 bytes/token here).
- Publication is one locked transition: complete groups, matching generations, live protections; a timed-out buffer stays quarantined.

# Moving and reusing state

# Shared paths, overlap, and when to stop waiting

- Shared resources bound transfers: two 1 KiB/ms NICs behind a 0.75 KiB/ms path give $T \geq 1 + 3/0.75 = 5$ ms; adaptive slices weigh load and locality. [DERIVED: book fixtures]

# Shared paths, overlap, and when to stop waiting

- Shared resources bound transfers: two 1 KiB/ms NICs behind a 0.75 KiB/ms path give $T \geq 1 + 3/0.75 = 5$ ms; adaptive slices weigh load and locality. [DERIVED: book fixtures]
- Two-slot layer streaming finishes at 14 ms versus 20 serial; a late reader delays it to 15 — release needs every reader.

# Shared paths, overlap, and when to stop waiting

- Shared resources bound transfers: two 1 KiB/ms NICs behind a 0.75 KiB/ms path give $T \geq 1 + 3/0.75 = 5$ ms; adaptive slices weigh load and locality. [DERIVED: book fixtures]
- Two-slot layer streaming finishes at 14 ms versus 20 serial; a late reader delays it to 15 — release needs every reader.
- Compare routes to the same endpoint: P2 remote reaches the first token at 12 ms at $\beta = 1$, but 21 ms at $\beta = 0.1$.

# Shared paths, overlap, and when to stop waiting

- Shared resources bound transfers: two 1 KiB/ms NICs behind a 0.75 KiB/ms path give $T \geq 1 + 3/0.75 = 5$ ms; adaptive slices weigh load and locality. [DERIVED: book fixtures]
- Two-slot layer streaming finishes at 14 ms versus 20 serial; a late reader delays it to 15 — release needs every reader.
- Compare routes to the same endpoint: P2 remote reaches the first token at 12 ms at $\beta = 1$, but 21 ms at $\beta = 0.1$.
- Prefetch stopping: 18/15/16 ms for best effort, timeout and wait complete — reversing to 28/23/16 when suffix costs change.

# Placement, admission and recovery

# From a cache hit to a completed request

- Mooncake's published heuristic scores one branch per worker; the book's finite comparator filters by a 15 ms predicate and may reject an empty set. [REPORTED: Mooncake Algorithm 1]

# From a cache hit to a completed request

- Mooncake's published heuristic scores one branch per worker; the book's finite comparator filters by a 15 ms predicate and may reject an empty set. [REPORTED: Mooncake Algorithm 1]
- Forecast decode at the arrival horizon: $3 - 2 + 4 = 5$ already fills a five-request limit; bytes need a separate atomic reservation.

# From a cache hit to a completed request

- Mooncake's published heuristic scores one branch per worker; the book's finite comparator filters by a 15 ms predicate and may reject an empty set. [REPORTED: Mooncake Algorithm 1]
- Forecast decode at the arrival horizon: $3 - 2 + 4 = 5$ already fills a five-request limit; bytes need a separate atomic reservation.
- Losing $B_2$ leaves a four-token fallback at 16 ms: defer or reject. Goodput counts only qualifying completions (40 requests/s on A/B/C).

# From a cache hit to a completed request

- Mooncake's published heuristic scores one branch per worker; the book's finite comparator filters by a 15 ms predicate and may reject an empty set. [REPORTED: Mooncake Algorithm 1]
- Forecast decode at the arrival horizon: $3 - 2 + 4 = 5$ already fills a five-request limit; bytes need a separate atomic reservation.
- Losing $B_2$ leaves a four-token fallback at 16 ms: defer or reject. Goodput counts only qualifying completions (40 requests/s on A/B/C).

Reported Mooncake gains keep their own models, hardware and workloads. Next: Chapter 44's KDA-Aware Prefix Caching and Long-Context Scheduling.